Rapport Projet RL - Agent Rolling Ball

Auteur: Ajax DESHAYES--HUET

Introduction

Contexte

Ce projet porte sur l'entrainement d'un agent de Reinforcement Learning dans un environnement Unity (ML-Agents), avec un controle discret des actions pour deplacer une balle vers une cible (cube).

L'agent est entraine avec un algorithme Deep Q-Network (DQN), implante en Python, qui interagit avec Unity via un wrapper Gym.

Objectifs

- Concevoir et completer un agent DQN fonctionnel.
- Etudier l'impact des hyperparametres et des recompenses sur l'apprentissage.
- Comparer 4 experiences avec configurations differentes.
- Interpreter les resultats (performance, stabilite, limites).
- Faire evoluer les objectifs au fil des experiences a partir des resultats observes.

Logique generale du rapport

Le rapport doit montrer une demarche iterative et non une suite de tests isoles.
Chaque experience sert a:

1. observer un comportement,
2. l'interpreter,
3. identifier une limite,
4. ajuster les parametres,
5. verifier si la nouvelle hypothese ameliore la situation.

Autrement dit, les resultats d'une experience modifient l'objectif de la suivante.

Methodologie

Rappel du cadre algorithmique

- Paradigme: Reinforcement Learning.
- Algorithme: DQN (Q-network + target network + replay memory).
- Fonction objectif: approximation de la Q-function.

Equation de Bellman (forme utilisee en DQN):

$$Q(s_t, a_t) \leftarrow r_t + \gamma \max_a Q_{target}(s_{t+1}, a)$$

Concepts importants a mobiliser

- **Reinforcement Learning**: apprentissage par interaction agent-environnement.
- **Equation de Bellman**: met a jour la valeur d'une action a partir du reward immediat + futur estime.

- **Gradient / descente de gradient**: ajustement des poids du reseau pour reduire la loss.

Parametres modifiables et hypotheses

- **Gamma (γ)**: poids des rewards futures.
- **N (nombre d'episodes)**: duree d'entrainement (risque sous-apprentissage si trop faible).
- **Learning rate (lr)**: vitesse de mise a jour des poids.
- **Epsilon (exploration/exploitation)**: comportement aleatoire vs politique apprise.
- **F (frequence de mise a jour target network)**.
- **B (batch size)**: taille d'echantillon replay pour chaque update.
- **M (buffer memory)**: capacite de replay memory.
- **NN (taille reseau)**: capacite de representation, cout de calcul, risque d'instabilite.
- **Rewards**: signal d'apprentissage (shaping + terminal rewards).

Experiences

> Consigne: lancer **4 experiences** avec des parametres differents.

Fil conducteur experimental

Le rapport peut etre raconte comme une suite d'iterations:

- Experience 1: base initiale apres completion des exercices.
- Experience 2: ajustement pour reduire le lag et accelerer l'experience.
- Experience 3: correction d'un sous-apprentissage ou d'une instabilite observee.
- Experience 4: affinement final, validation de la configuration retenue et reduction des spikes encore trop presents.

Tableau recapitulatif des experiences

Experience	Gamma	N episodes	lr	Epsilon (start/end/decay)	F	B	M	NN	Rewards	Attente principale
E1 (baseline)	0.99	250	1e-4	0.9 / 0.05 / 2000	500	256	100000	[9,512,512,5]	shaping +/-0.02, terminal +1/-1, timeout -0.5	Baseline stable a lancer
E2	0.99	150	7e-5	0.9 / 0.05 / 1500	500	128	100000	[9,512,512,5]	shaping +/-0.01, terminal +1/-1, timeout -0.7, max steps 400	Reduire le temps de run et les spikes tout en limitant le risque de "circling"
E3	0.99	220	1e-4	0.9 / 0.02 / 2000	300	128	100000	[9,512,512,5]	shaping +/-0.01, terminal +1/-1, timeout -0.5, max	Recuperer stabilite et taux de succes sans changer l'architecture

Experience	Gamma	N episodes	lr	Epsilon (start/end/decay)	F	B	M	NN	Rewards	Attente principale
									steps 500	
E4 (finale)	0.99	250	7e-5	0.9 / 0.02 / 2000	300	128	100000	[9,512,512,5]	shaping +/-0.01, terminal +1/-1, timeout -0.5, max steps 500	Valider le meilleur compromis et lisser les spikes

Detail de chaque experience (a dupliquer)

Experience E1 - Baseline de reference

- **Choix des parametres**:
  - Configuration actuelle du projet (baseline): `gamma=0.99` , `N=250` , `lr=1e-4` , `epsilon=0.9/0.05/2000` , `F=500` , `batch_size=256` , `memory=100000` , reseau `[9,512,512,5]` .
  - Reward design: shaping `+0.02/-0.02` , terminal `+1/-1` , timeout `-0.5` .
- **Methodologie / reflexion / approche**:
  - Cette experience sert de point de reference pour comparer toutes les experiences suivantes.
- **Attentes avant execution**:
  - Verifier la stabilite globale de la courbe reward et la tendance de duree moyenne des episodes.
- **Resultats observes**:
  - Le run est complet: `episodes_completed=250` , `total_steps=47564` , `run_total_seconds=1247.941` .
  - Les moyennes globales sont positives: `mean_episode_reward=1.778` , `mean_episode_duration=190.256` .
  - En execution locale (Unity Editor), des ralentissements ont ete observes sur la machine pendant certains passages de l'entrainement.
  - La courbe reward monte progressivement en debut d'entrainement, puis se stabilise avec une moyenne lissee positive et des pics residuels.
  - La duree des episodes baisse globalement, mais reste tres variable avec des pics tout au long du run.
  - Des ecarts brusques apparaissent entre episodes consecutifs (ex. episodes 12-15), ce qui est compatible avec l'alea du spawn de la cible et l'exploration epsilon-greedy.
  - Attention d'interpretation: le label du graphe indique "Last reward", mais la serie tracee correspond en pratique a la reward cumulee par episode.
- **Interpretation rapide**:
  - E1 valide la base DQN: l'agent apprend a atteindre la cible plus regulierement en fin de run.
  - Les spikes ne signifient pas un bug en soi; ils sont attendus dans un environnement stochastique avec exploration.
  - Le risque principal a surveiller est le "reward shaping farming" (l'agent gagne des points en tournant autour de la cible), ce qui motive une reduction du shaping et un timeout plus penalise en E2.

Experience E2 - Compromis vitesse/stabilite

- **Choix des parametres**:
  - Hyperparametres Python: `gamma=0.99` , `N=150` , `lr=7e-5` , `epsilon=0.9/0.05/1500` , `F=500` , `batch_size=128` , `memory=100000` , reseau `[9,512,512,5]` .
  - Reward design Unity: shaping `+0.01/-0.01` (au lieu de `+/-0.02`), terminal `+1/-1` conserve, timeout `-0.7` (au lieu de `-0.5`) et limite episode `400` steps (au lieu de `500`).
  - Motivation principale: pendant E1, la simulation Unity a montre des ralentissements (lag), donc E2 privilegie un compromis vitesse/stabilite avant de pousser la performance brute.
- **Methodologie / reflexion / approche**:

- E2 cible trois objectifs operationnels: reduire le temps de simulation, attenuer les oscillations, et limiter les comportements opportunistes de type "tourner autour de la cible".
- Les changements sont volontairement moderes pour rester comparables a E1 et isoler les effets principaux.
- Justification des changements lies au lag:
  - `N=150` reduit directement la duree totale du run et accelere le cycle test-analyse.
  - `batch_size=128` diminue le cout de chaque mise a jour DQN (moins de calcul par backward pass), ce qui aide a limiter la charge CPU/GPU pendant l'entrainement dans l'Editor Unity.
  - `max steps=400` evite des episodes tres longs peu informatifs, ce qui reduit aussi le temps de simulation.

- **Attentes avant execution**:
  - Diminution du temps total de run grace a `N` plus faible, `batch_size` plus petit et episodes plus courts.
  - Courbe reward potentiellement un peu moins explosive (moins de points shaping par step + `lr` plus faible).
  - Baisse des episodes longs sans succes terminal, grace au timeout plus strict et plus penalise.
  - Verification attendue du lag: baisse de `run_total_seconds` et de `mean_episode_elapsed_seconds` par rapport a E1.
- **Resultats observes**:
  - Run complet: `episodes_completed=150` , `total_steps=41502` , `run_total_seconds=842.105` .
  - Moyennes globales: `mean_episode_reward=0.384` , `mean_episode_duration=276.68` , `mean_episode_elapsed_seconds=5.574` .
  - Les spikes restent importants sur la reward et sur la duree; la stabilisation est moins nette que prevu.
  - Beaucoup d'episodes se terminent au timeout (`terminal_reward=-0.7`): 59/150 (39.33%).
  - Le succes existe mais reste partiel (`terminal_reward=1.0`): 63/150 (42.00%).
  - Observation pratique post-run: une partie du lag percu venait du contexte d'execution (focus de la fenetre terminal), pas uniquement des hyperparametres.
- **Interpretation rapide**:
  - E2 a bien reduit le temps total de run par rapport a E1, mais au prix d'une baisse de qualite d'apprentissage (reward moyenne plus faible, episodes plus longs, nombreux timeouts).
  - La combinaison `eps_decay=1500` + `max steps=400` + `timeout=-0.7` parait trop contraignante pour converger proprement.
  - Le lag reste contraignant en pratique, donc E3 conserve un compromis `batch_size=128` et corrige d'abord les autres facteurs (timeout, epsilon final, nombre d'episodes).

Experience E3 - Stabilisation apres E2

- **Choix des parametres**:
  - Hyperparametres Python: `gamma=0.99` , `N=220` , `lr=1e-4` , `epsilon=0.9/0.02/2000` , `F=300` , `batch_size=128` , `memory=100000` , reseau `[9,512,512,5]` .
  - Reward design Unity: shaping `+0.01/-0.01` conserve (anti-circling), terminal `+1/-1` conserve, timeout `-0.5` et limite episode `500` steps.
- **Methodologie / reflexion / approche**:
  - E3 corrige E2 avec une strategie conservative: ne pas toucher l'architecture du reseau, conserver `eps_decay=2000` et retenir un compromis de calcul avec `B=128` pour eviter les ralentissements severes observes a `B=256` .
  - Le parametre `F` est abaisse de `500` a `300` pour synchroniser plus frequemment le target network, afin de reduire certaines oscillations observees en E2 sans introduire une rupture majeure de configuration.
  - `N` passe a `220` pour donner plus de temps d'apprentissage que E2 (150 episodes) sans revenir au cout complet de E1 (250 episodes).
  - `eps_end` passe de `0.05` a `0.02` pour reduire l'aleatoire en fin d'entrainement et stabiliser la politique finale.
  - Le but est d'isoler l'effet des contraintes trop strictes d'E2 (timeout court et penalite forte) qui ont probablement augmente les echecs a 400 steps.
- **Attentes avant execution**:
  - Augmentation du taux de succes et baisse du taux de timeout.
  - Courbe reward plus lisible (moins de saturation autour des episodes coupes a 400).
  - Duree de run raisonnable (N=220) tout en laissant plus de temps d'apprentissage que E2.
- **Resultats observes**:
  - Le run est complet: `episodes_completed=220` , `total_steps=54110` , `run_total_seconds=1481.531` .
  - Moyennes globales: `mean_episode_reward=0.5226` , `mean_episode_duration=245.95` , `mean_episode_elapsed_seconds=6.696` .
  - Repartition des terminaisons:

        - succes (`terminal_reward=1.0`): 123/220 (55.9%).
        - timeout (`terminal_reward=-0.5`): 37/220 (16.8%).
        - echec (`terminal_reward=-1.0`): 60/220 (27.3%).
    - La courbe reward progresse globalement mais reste volatile, avec des spikes encore visibles.
- **Interpretation rapide**:
    - E3 corrige une partie des effets negatifs observes en E2.
    - Le taux de succes remonte nettement (55.9% vs 42.0% en E2) et les timeouts baissent fortement (16.8% vs 39.33% en E2).
    - E3 reste toutefois en dessous de E1 sur la performance globale (reward moyenne et regularite de convergence).
    - Le compromis E3 est valide pour stabiliser sans exploser le cout de calcul, mais ne depasse pas encore la baseline E1.

Experience E4 - Finale (validation du compromis)

- **Choix des parametres**:
    - Configuration finale de validation: `gamma=0.99` , `N=250` , `lr=7e-5` , `epsilon=0.9/0.02/2000` , `F=300` , `batch_size=128` , `memory=100000` , reseau `[9,512,512,5]` .
    - Reward design Unity: shaping `+0.01/-0.01` , terminal `+1/-1` , timeout `-0.5` , limite episode `500` steps.
- **Methodologie / reflexion / approche**:
    - Cette experience sert de validation finale de la meilleure configuration candidate, avec un apprentissage un peu plus prudent pour lisser les spikes encore trop presents.
- **Attentes avant execution**:
    - Verifier si une baisse du learning rate et un temps d'apprentissage plus long reduisent les oscillations sans detruire le taux de succes.
- **Resultats observes**:
    - Le run final est complet: `episodes_completed=250` , `total_steps=63588` , `run_total_seconds=1396.504` .
    - Moyennes globales: `mean_episode_reward=0.9013` , `mean_episode_duration=254.352` , `mean_episode_elapsed_seconds=5.5687` .
    - Repartition des terminaisons:
        - succes (`terminal_reward=1.0`): 152/250 (60.8%).
        - timeout (`terminal_reward=-0.5`): 39/250 (15.6%).
        - echec (`terminal_reward=-1.0`): 59/250 (23.6%).
    - La courbe reward reste variable, mais la fin de run est plus lisible que E3 avec moins de spikes marquants.
- **Interpretation rapide**:
    - E4 ameliore nettement E3 sur la reward moyenne et le taux de succes, tout en conservant un taux de timeout bas.
    - Le compromis vise (stabilite sans effondrement des performances) est globalement atteint.
    - E4 ne depasse pas E1 en performance brute, mais offre un equilibre plus prudent et plus robuste que E2/E3.

Resultats

Cette section sert de synthese globale. Les details d'observation et d'interpretation sont documentes dans chaque experience.

Resultats bruts

- Figure 1: [courbe reward par episode]
- Figure 2: [courbe duree par episode]
- Figure 3 (optionnel): [moving average ou taux de succes]

Resultats attendus vs observes (synthese)

- **Ce qui etait attendu**:
    - E2 devait reduire le temps de run tout en conservant une stabilite acceptable de l'apprentissage.
    - Le nombre de spikes devait diminuer avec un shaping plus faible et un lr plus prudent.
- **Ce qui a ete observe**:
    - Le temps total a bien baisse par rapport a E1, mais la qualite d'apprentissage a recule en moyenne sur E2.
    - Les spikes restent forts sur E2 et les episodes timeout restent frequents.

- E3 recupere une partie de la stabilite perdue en E2 (succes en hausse, timeouts en baisse), sans retrouver le niveau global de E1.
- E4 confirme une amelioration supplementaire vs E3 (reward moyenne 0.901 vs 0.523, succes 60.8% vs 55.9%), avec une fin de run visuellement moins erratique.

- **Ecarts et surprises**:
  - Le couple timeout court/penalite forte semble avoir trop contraint la convergence; l'effet de `batch_size` ne doit pas etre sur-interprete sans isoler les changements.
  - Le lag percu depend aussi du contexte d'execution (focus terminal), pas seulement des hyperparametres RL.

Tableau de comparaison finale

Critere	E1	E2	E3	E4
Reward moyenne (fin de run)	1.778	0.384	0.523	0.901
Reward moyenne lissee	Positive, stable en fin de run	Proche de 0, instable	Positive mais volatile (amelioration vs E2, inferieure a E1)	Positive, plus reguliere en fin de run (amelioration vs E3)
Duree moyenne episode	190.256	276.68	245.95	254.352
Stabilite (spikes)	Moyenne	Faible (spikes frequents)	Moyenne-faible (amelioration vs E2)	Moyenne (moins de spikes en fin de run vs E3)
Taux de succes (si mesure)	72.4%	42.0%	55.9%	60.8%

Analyse

Cette section propose une lecture transversale E1->E4, en complement des analyses locales deja faites dans chaque experience.

Tendances identifiees

- E1 fournit la meilleure base de stabilite globale dans l'etat actuel des tests.
- E2 montre qu'une reduction simultanee de plusieurs contraintes (B plus petit, timeout plus court et plus penalise, eps_decay plus rapide) peut degrader la qualite d'apprentissage.
- E4 ameliore le compromis obtenu en E3: meilleure reward moyenne, meilleur taux de succes, et fin de run moins erratique.
- La sensibilite aux conditions de run (focus terminal / charge machine) est suffisante pour impacter le runtime et possiblement la perception de stabilite.
- Le shaping reduit (+/-0.01) limite le risque de reward farming, mais ne suffit pas seul a garantir une convergence propre.

Limites

- Taille de l'echantillon: encore limitee (un run principal par configuration).
- Variabilite due a l'alea: seed non fixee et spawn cible aleatoire.
- Sensibilite au reward shaping: forte, notamment sur l'equilibre entre guidance dense et succes terminal.
- Temps de calcul / vitesse simulation: contrainte importante en execution CPU locale.
- Certaines ameliorations de vitesse peuvent detruire de la stabilite, et inversement.

Lecture des spikes

- Des spikes ne signifient pas automatiquement un bug.
- Ils peuvent venir de l'exploration encore trop forte, d'un shaping trop agressif, ou d'une politique pas encore stabilisee.

- Il faut lire ensemble reward moyenne, duree moyenne, et taux de succes.

Menaces a la validite

- Difference entre entrainement en Editor et build headless.
- Parametres non isoles si plusieurs changent a la fois.
- Dependance a l'initialisation aleatoire de l'environnement.

Conclusion

- **Synthese des resultats**:
  - E1 reste la reference la plus stable a ce stade, avec une reward moyenne nettement positive et une duree d'episode plus faible.
  - E2 a bien atteint l'objectif de reduction du temps de run, mais avec une degradation de la qualite d'apprentissage et une forte variabilite.
  - E3 est defini comme une correction ciblant la stabilite sans changer l'architecture reseau.
  - E4 valide le compromis final: progression nette par rapport a E2/E3, et fin de run plus stable, sans toutefois depasser E1 en performance brute.
- **Reponse aux objectifs initiaux**:
  - Objectif 1 (agent DQN fonctionnel): atteint.
  - Objectif 2 (etudier l'impact des parametres): atteint, avec un effet clair des changements sur succes, timeouts et stabilite.
  - Objectif 3 (comparaison multi-experiences): atteint, les 4 experiences ont ete executees et comparees.
- **Configuration recommandee**:
  - Pour un compromis performance/stabilite: configuration E4 (`gamma=0.99` , `N=250` , `lr=7e-5` , `epsilon=0.9/0.02/2000` , `F=300` , `batch_size=128` , shaping `+/-0.01` , timeout `-0.5` , max steps `500`).
  - Pour la meilleure performance brute observee sur ce projet: configuration E1.
- **Ameliorations futures**:
  - Evaluer E3 sur au moins 2 runs pour mesurer la robustesse.
  - Fixer une seed pour reduire la variance inter-runs.
  - Tester une evolution algorithmique legere (Double DQN) si la stabilite reste insuffisante.
  - Passer sur build Unity plus leger si possible pour reduire l'impact des contraintes machine.

Phrase de synthese type

L'analyse doit montrer comment chaque experience modifie la suivante: le projet est donc un processus d'optimisation iterative, pas seulement une comparaison de chiffres.